

DAMM v2

Release 0.2.2

Smart Contract Security Assessment

June 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Reward Delegate Can Clear Rewards Without Transfer	6
4.2	Delegate Permissions Can Be Reused After Delegate Or Owner Changes	7
4.3	Position Lock Permission Lacks Granularity	8
4.4	Informational and Undetermined Issues	8
5	Disclaimer	10



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *DAMM v2* smart contracts, starting on June 16th, 2026, and concluding on June 22th, 2026.

Project Overview

Meteora DAMM v2 Release 0.2.2 introduces delegated position management for selected liquidity, fee, reward, and locking actions. Position owners can set a permission mask that works with token delegate approvals, while closing and splitting positions remain owner controlled. The release also updates owner attribution in position events, tightens reward vault token validation, and derives permission limits from the operator permission model.

Audit Scope

The assessment scope contains mainly the smart contracts of the cp-amm program for the *DAMM v2* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DAMM v2
 - Codebase: <https://github.com/MeteoraAg/damm-v2>
 - Release 0.2.2: [PR-210](#)
 - Commit Hash: 1e8580ffa3a98fdbdc75ac66bc3082f5c27bb51e

The issues described in this report have been resolved by the following commit:

- DAMM v2
 - Codebase: <https://github.com/MeteoraAg/damm-v2>
 - Commit Hash: 46179ccf7ecc9d159746564288bd8b28fc14da82

We listed the files we have audited below:

- DAMM v2 Release 0.2.2
 - `libs/derive-variant-count/src/lib.rs`
 - `programs/cp-amm/src/constants.rs`
 - `programs/cp-amm/src/error.rs`
 - `programs/cp-amm/src/event.rs`
 - `programs/cp-amm/src/instructions/admin/ix_create_operator_account.rs`
 - `programs/cp-amm/src/instructions/ix_add_liquidity.rs`
 - `programs/cp-amm/src/instructions/ix_claim_position_fee.rs`
 - `programs/cp-amm/src/instructions/ix_claim_reward.rs`
 - `programs/cp-amm/src/instructions/ix_lock_inner_position.rs`
 - `programs/cp-amm/src/instructions/ix_lock_position.rs`
 - `programs/cp-amm/src/instructions/ix_permanent_lock_position.rs`



- `programs/cp-amm/src/instructions/ix_remove_liquidity.rs`
- `programs/cp-amm/src/instructions/ix_split_position2.rs`
- `programs/cp-amm/src/instructions/ix_update_delegate_permission.rs`
- `programs/cp-amm/src/instructions/mod.rs`
- `programs/cp-amm/src/lib.rs`
- `programs/cp-amm/src/state/operator.rs`
- `programs/cp-amm/src/state/position.rs`

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 1 medium issues
- 2 low issues
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Reward Delegate Can Clear Rewards Without Transfer	Medium	Fixed
02	Delegate Permissions Can Be Reused After Delegate Or Owner Changes	Low	Acknowledged
03	Position Lock Permission Lacks Granularity	Low	Acknowledged
04	Incomplete Delegate Endpoint List	Informational	Fixed
05	Delegate Actions Are Not Recorded In Events	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Reward Delegate Can Clear Rewards Without Transfer

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In `claim_reward` IX, a position NFT delegate can pass the authority check with either `ClaimReward` or `ClaimRewardToOwner`.

After the authority check, the instruction clears the pending reward amount by calling `position.claim_reward`. If the reward vault is frozen and `skip_reward == 1`, the instruction succeeds without transferring reward tokens.

```
98 // get all pending reward
99 let total_reward = position.claim_reward(index)?;
100
101 // transfer rewards to user
102 if total_reward > 0 {
103     if ctx.accounts.reward_vault.is_frozen() {
104         require!(skip_reward == 1,
105             PoolError::RewardVaultFrozenSkipRequired)
106     } else {
```

[programs/cp-amm/src/instructions/ix_claim_reward.rs#L98-L105](#)

However, this means a delegate that has `ClaimRewardToOwner` permission but not `ClaimReward` permission can do more than claim rewards to the owner's token account. When the reward vault is frozen, that delegate can set `skip_reward == 1` and clear the owner's pending reward amount without any reward transfer.

Impact

A position NFT delegate with only `ClaimRewardToOwner` permission can clear the position owner's pending rewards when the reward vault is frozen. This exceeds the expected owner destination only reward claim permission.

Recommendation

It is recommended to reject delegates that only have `ClaimRewardToOwner` permission when the reward vault is frozen and `skip_reward == 1`. A delegate with `ClaimRewardToOwner` should not be able to clear pending rewards unless reward tokens are transferred to the owner's token account.



Mitigation Review Log

Fixed in the commit 706a92c72908af842317dfa69e96e70ab808d29c.

4.2 Delegate Permissions Can Be Reused After Delegate Or Owner Changes

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Design

Description

When a delegate calls a position instruction, the program checks the current token account delegate and then checks with `Position.delegate_permission`. Since this field is not tied to a specific owner or delegate, any current zero-amount SPL delegate for the position NFT can use whatever permission bits are already stored on the position.

```
559     self.assert_signer_is_delegate(nft_token_account, signer)?;  
560  
561     require!(  
562         self.is_delegate_permission_allowed(permission),  
563         PoolError::InvalidPermission  
564     );
```

[programs/cp-amm/src/state/position.rs#L559-L564](#)

This means the permission can `outlive` the authorization it was meant to represent.

- If the owner changes the SPL delegate from A to B, B inherits the existing `delegate_permission` bits.
- If the position NFT is transferred, the permission bits also remain on the Position. A delegate on the new owner's token account can then inherit permissions that were configured before the transfer.

Impact

Stale permission bits can authorize a different delegate than the owner originally intended. After the position NFT is transferred, permissions set by the previous owner can also apply to the new owner's delegate.

Recommendation

It is recommended to bind the permission to the delegate it was granted to, and also invalidate it when the position NFT owner changes. Since the Position account does not have enough padding for this metadata, the binding can be stored in a separate authorization PDA. When a delegate calls a position instruction, the program should reject the call if the current position NFT owner or delegate no longer matches the stored authorization.



Mitigation Review Log

Meteora Team: This is an acceptable tradeoff since we don't have the space to add a Pubkey without increasing the size of the Position account.

4.3 Position Lock Permission Lacks Granularity

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Design

Description

The delegate permission model uses a single `LockPosition` permission for `lock_position`, `lock_inner_position`, and `permanent_lock_position`. These instructions do not represent the same level of authority: `lock_position` and `lock_inner_position` create vesting locks, while `permanent_lock_position` permanently moves unlocked liquidity into permanent locked liquidity.

Because all three instructions check the same permission, an owner cannot allow a delegate to create regular locks without also allowing that delegate to permanently lock liquidity.

Impact

A delegate granted lock authority can use every lock instruction, including `permanent_lock_position`. The owner cannot configure separate delegate permissions for temporary vesting locks and permanent locks.

Recommendation

It is recommended to `split lock permissions` by action. Permanent locking should require a distinct permission bit.

Mitigation Review Log

Meteora Team: Intentional part of the design to combine the actions under one permission.

4.4 Informational and Undetermined Issues

Incomplete Delegate Endpoint List

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA



The delegate documentation in `CHANGELOG.md` lists `remove_liquidity` as delegate-enabled, but does not list `remove_all_liquidity`. Since `remove_all_liquidity` is also an instruction that can be called by a valid remove-liquidity delegate, it should be included in the delegate-enabled instruction list and related error-change documentation.

Delegate Actions Are Not Recorded In Events

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Code QA

Delegate-enabled position instructions emit events that record the `position owner` but not the actual signer. As a result, delegate calls can appear as owner actions to other components that rely on events.

It is recommended to include the actual signer, or emit a new event version that distinguishes owner calls from delegate calls.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

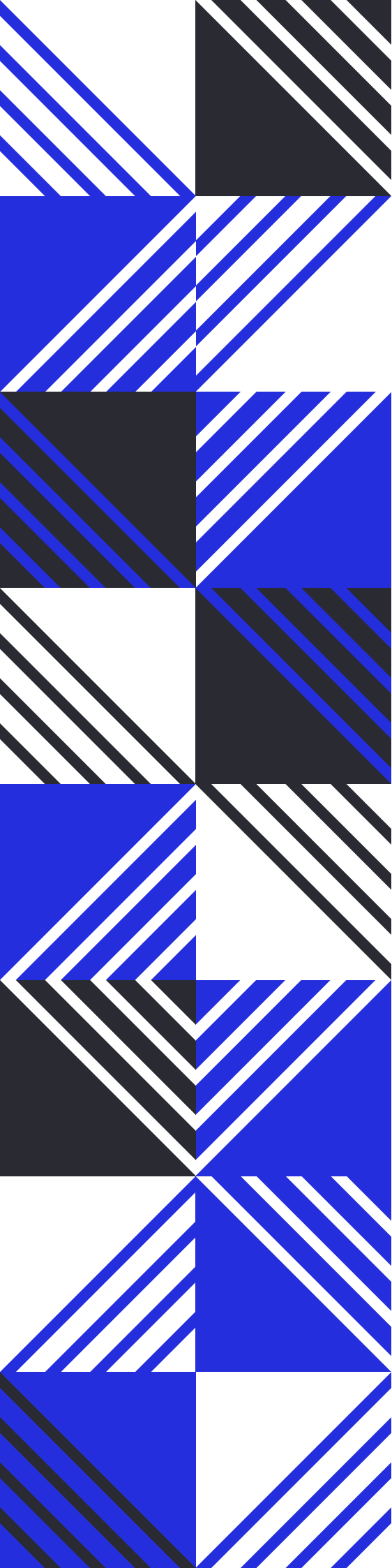
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs